

```

22     {
23         "field": "profile.company_data.recent_funding",
24         "operator": "=",
25         "value": true,
26         "points": 20,
27         "reason": "Recently funded companies more likely to buy"
28     }
29 ]
30 }
31 ↓
32
33 3. SegmentationLogicSkill
34 ↓
35 await skill.use_tool("update_segmentation_criteria",
36     segmentation_id=segment_id,
37     user_id=user_id,
38     criteria_updates=request_data
39 )
40 ↓
41 - Merges new criteria with existing
42 - Stores in segmentations.criteria JSONB
43 - Logs in user_feedback table
44 ↓
45
46 4. Background Job
47 ↓
48 Trigger: Recalculate scores for all contacts in this segment
49 ↓
50 For each contact:
51     - SegmentCalculatorAgent.run(contact_id, [segment_id])
52     - Re-evaluate with new criteria
53     - Update scores
54 ↓
55
56 5. User sees updated results
57 ↓
58 Dashboard shows: "127 contacts now match (was 98)"
59 "Avg fit score: 84 (was 78)"

```

API Implementation:

Code block

```

1  @router.post("/segmentations/{segmentation_id}/update-criteria")
2  async def update_segmentation_criteria(
3      segmentation_id: str,
4      criteria_updates: dict,
5      recalculate_now: bool = True,
6      current_user: User = Depends(get_current_user)
7  ):
8      """
9      Update segmentation criteria with user input
10     """
11
12     # Use SegmentationLogicSkill
13     skill = SegmentationLogicSkill(db_pool=db)
14
15     updated = await skill.use_tool(
16         "update_segmentation_criteria",
17         segmentation_id=segmentation_id,
18         user_id=current_user.id,

```

```
19         criteria_updates=criteria_updates
20     )
21
22     # Trigger recalculation (background job)
23     if recalculate_now:
24         await trigger_segment_recalculation(segmentation_id)
25
26     return {
27         "segmentation_id": segmentation_id,
28         "criteria_updated": True,
29         "recalculation_triggered": recalculate_now,
30         "updated_criteria": updated["updated_criteria"]
31     }
```

4.2 Workflow: User Validates AI Insight

Scenario: AI suggests pain point, user confirms with details

Flow:

Code block

```
1  1. User sees AI insight
2      ↓
3  Contact Profile shows:
4      💡 AI Insight (85% confidence)
5      "Pain point: Manual data entry consuming team time"
6
7      [Confirm] [Reject] [Add Details]
8      ↓
9
10 2. User clicks "Confirm" + adds details
11     ↓
12     Modal appears:
13     "Confirm this insight:"
14
15     Exact quote: "We're drowning in manual work"
16     Severity: [Critical ▼]
17     Time impact: "15 hours/week"
18     Budget allocated: [Yes ✓]
19
20     [Save]
21     ↓
22
23 3. API Call
24     ↓
25     POST /api/contacts/{id}/insights/validate
26     {
27         "insight_type": "pain_point",
28         "insight_text": "Manual data entry consuming team time",
29         "validation": "confirmed",
30         "confirmed_data": {
31             "pain_point": "Team spends 15 hours/week on manual data
32 entry",
33             "exact_quote": "We're drowning in manual work",
34             "severity": "Critical",
35             "budget_allocated": true
36         }
37     }
38     ↓
```

```
38
39 4. FeedbackCaptureSkill
40   ↓
41   await skill.use_tool("capture_insight_validation", ...)
42   ↓
43   - Moves insight from ai_insights → confirmed_facts
44   - Updates insight_validation tracking
45   - Stores feedback in user_feedback table
46   ↓
47
48 5. Learning Loop
49   ↓
50   - Track: AI confidence was 85%, user confirmed
51   - Pattern: "Manual work" keyword → high accuracy
52   - Adjust: Increase confidence for similar patterns
53   ↓
54
55 6. UI Updates
56   ↓
57   ✓ Confirmed Fact
58   "Team spends 15 hours/week on manual data entry"
59   Source: Confirmed by you on Dec 16, 2024
```

API Implementation:

Code block

```
1  @router.post("/contacts/{contact_id}/insights/validate")
2  async def validate_insight(
3      contact_id: str,
4      validation_request: InsightValidationRequest,
5      current_user: User = Depends(get_current_user)
6  ):
7      """
8      Validate AI-generated insight
9      """
10
11     skill = FeedbackCaptureSkill(db_pool=db)
12
13     result = await skill.use_tool(
14         "capture_insight_validation",
15         user_id=current_user.id,
16         contact_id=contact_id,
17         insight_type=validation_request.insight_type,
18         insight_text=validation_request.insight_text,
19         validation=validation_request.validation,
20         confirmed_data=validation_request.confirmed_data
21     )
22
23     return {
24         "status": "success",
25         "insight_moved": result["insight_moved"],
26         "learning_triggered": True
27     }
```

4.3 Workflow: User Adjusts Relationship Score

Scenario: AI calculates relationship strength = 75, user knows it's actually 90

Flow:

Code block

```
1  1. User sees score
2    ↓
3    Relationship Strength: 75
4    [Adjust Score]
5    ↓
6
7  2. User adjusts
8    ↓
9    Modal:
10   "Adjust Relationship Strength"
11
12   AI Score: 75
13   Your Score: [90] (slider)
14
15   Reason: "Just closed a big deal, relationship is stronger"
16
17   [Save]
18   ↓
19
20  3. API Call
21    ↓
22    POST /api/contacts/{id}/adjust-score
23    {
24      "field_path": "relationship.strength_score",
25      "original_score": 75,
26      "adjusted_score": 90,
27      "reason": "Just closed a big deal"
28    }
29    ↓
30
31  4. FeedbackCaptureSkill
32    ↓
33    await skill.use_tool("capture_score_adjustment", ...)
34    ↓
35    - Updates contact.profile.relationship.strength_score = 90
36    - Stores feedback in user_feedback table
37    - Marks: User consistently adjusts scores +15 for deal wins
38    ↓
39
40  5. Learning
41    ↓
42    - Pattern detected: After deal close → score +10-20
43    - Adjust: AI should boost score when deal closes
44    ↓
45
46  6. UI Updates
47    ↓
48    Relationship Strength: 90 ★ (User adjusted)
49    Reason: "Just closed a big deal"
```

4.4 Workflow: User Adds Custom Fact

Scenario: User learns something not detected by AI

Flow:

```
1 1. User adds fact
2   ↓
3   Contact Profile → [+ Add Fact]
4   ↓
5
6 2. Modal appears
7   ↓
8   "Add Fact About John Doe"
9
10  Type: [Goal ▼]
11
12  Goal: "Launch new product by Q2 2025"
13  Timeline: "6 months"
14  Executive mandate: [Yes ✓]
15  Source: "Told me in call today"
16
17  [Save]
18  ↓
19
20 3. API Call
21  ↓
22  POST /api/contacts/{id}/add-fact
23  {
24    "fact_type": "goal",
25    "fact_data": {
26      "goal": "Launch new product by Q2 2025",
27      "timeline": "6 months",
28      "executive_mandate": true,
29      "source": "Told me in call today"
30    }
31  }
32  ↓
33
34 4. FeedbackCaptureSkill
35  ↓
36  await skill.use_tool("capture_custom_fact", ...)
37  ↓
38  - Adds directly to confirmed_facts.goals
39  - Marks as user_added: true
40  - No AI validation needed
41  ↓
42
43 5. UI Updates
44  ↓
45  ✓ Confirmed Goal
46  "Launch new product by Q2 2025"
47  Added by you on Dec 16, 2024
```

2.5 DATABASE SCHEMA

5.1 Core Tables (Extend Existing)

No changes to existing tables! Just extend JSONB columns.

5.2 New Tables

Code block

```
1  -- interactions table
```

```

2 CREATE TABLE interactions (
3     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4     user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
5     contact_id UUID NOT NULL REFERENCES contacts(id) ON DELETE
6 CASCADE,
7     campaign_id UUID REFERENCES campaigns(id) ON DELETE SET NULL,
8     interaction_type VARCHAR(50) NOT NULL,
9     details JSONB NOT NULL DEFAULT '{} '::jsonb,
10    ai_analysis JSONB DEFAULT '{} '::jsonb,
11
12    occurred_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
13    created_at TIMESTAMPTZ DEFAULT NOW()
14 );
15
16 CREATE INDEX idx_interactions_contact ON interactions(contact_id);
17 CREATE INDEX idx_interactions_occurred ON interactions(occurred_at
18 DESC);
19
20 -- segmentations table
21 CREATE TABLE segmentations (
22     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
23     user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
24     name VARCHAR(200) NOT NULL,
25     description TEXT,
26     priority INTEGER DEFAULT 5 CHECK (priority >= 1 AND priority <=
27 10),
28     criteria JSONB NOT NULL DEFAULT '{} '::jsonb,
29     icp_definition JSONB DEFAULT '{} '::jsonb,
30
31     is_active BOOLEAN DEFAULT true,
32     created_at TIMESTAMPTZ DEFAULT NOW()
33 );
34
35 -- contact_segment_scores table
36 CREATE TABLE contact_segment_scores (
37     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
38     contact_id UUID NOT NULL REFERENCES contacts(id) ON DELETE
39 CASCADE,
40     segmentation_id UUID NOT NULL REFERENCES segmentations(id) ON
41 DELETE CASCADE,
42     fit_score INTEGER NOT NULL CHECK (fit_score >= 0 AND fit_score <=
43 100),
44     score_breakdown JSONB DEFAULT '{} '::jsonb,
45     scored_at TIMESTAMPTZ DEFAULT NOW(),
46     UNIQUE(contact_id, segmentation_id)
47 );
48
49 -- user_feedback table (NEW - for human-in-the-loop)
50 CREATE TABLE user_feedback (
51     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
52     user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
53     feedback_type VARCHAR(50) NOT NULL,
54     entity_type VARCHAR(50) NOT NULL,
55     entity_id UUID NOT NULL,
56
57     feedback_data JSONB NOT NULL DEFAULT '{} '::jsonb,

```

```

58
59     applied BOOLEAN DEFAULT false,
60     applied_at TIMESTAMPTZ,
61
62     created_at TIMESTAMPTZ DEFAULT NOW()
63 );
64
65 CREATE INDEX idx_user_feedback_user ON user_feedback(user_id);
66 CREATE INDEX idx_user_feedback_type ON
67     user_feedback(feedback_type);
68
69 CREATE INDEX idx_user_feedback_entity ON
70     user_feedback(entity_type, entity_id);
71
72 -- agent_activity_log table
73
74 CREATE TABLE agent_activity_log (
75     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
76     user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
77
78     agent_type VARCHAR(100) NOT NULL,
79     action VARCHAR(100) NOT NULL,
80     entity_type VARCHAR(50),
81     entity_id UUID,
82
83     input_data JSONB DEFAULT '{}'::jsonb,
84     output_data JSONB DEFAULT '{}'::jsonb,
85
86     ai_model VARCHAR(100),
87     confidence DECIMAL(3,2),
88     processing_time_ms INTEGER,
89
90     status VARCHAR(20) NOT NULL,
91     error_message TEXT,
92
93     created_at TIMESTAMPTZ DEFAULT NOW()
94 );
95
96 CREATE INDEX idx_agent_activity_type ON
97     agent_activity_log(agent_type);
98
99 CREATE INDEX idx_agent_activity_created ON
100     agent_activity_log(created_at DESC);

```

2.6 IMPLEMENTATION TASKS

EPIC 1: Database & Infrastructure

TASK-001: Create New Tables

- ☐ Create `interactions` table with indexes
- ☐ Create `segmentations` table with JSONB indexes
- ☐ Create `contact_segment_scores` table
- ☐ Create `user_feedback` table
- ☐ Create `agent_activity_log` table
- ☐ Test migrations on staging

TASK-002: Add JSONB Indexes to Existing Tables

- ☐ Add GIN indexes to `contacts.profile`

- ☐ Add specific indexes for common query paths
- ☐ Add full-text search indexes
- ☐ Performance test with 10K contacts

TASK-003: Redis Setup

- ☐ Connect to Supabase Redis
- ☐ Create RediSearch indexes
- ☐ Test vector similarity search
- ☐ Implement connection pooling

EPIC 2: Shared Skills Library

TASK-004: Build ContactQuerySkill

- ☐ Implement `get_contact` tool
- ☐ Implement `filter_contacts` with JSONB queries
- ☐ Implement `search_contacts_semantic` with Redis
- ☐ Write unit tests (>80% coverage)
- ☐ Performance test: <50ms for filters

TASK-005: Build SegmentationLogicSkill

- ☐ Implement `evaluate_contact_fit` tool
- ☐ Support user-defined filter criteria
- ☐ Support user-defined scoring rules
- ☐ Implement `update_segmentation_criteria` tool
- ☐ Write unit tests

TASK-006: Build FeedbackCaptureSkill

- ☐ Implement `capture_score_adjustment` tool
- ☐ Implement `capture_insight_validation` tool
- ☐ Implement `capture_custom_fact` tool
- ☐ Implement `get_feedback_summary` tool
- ☐ Write unit tests

TASK-007: Build ScoringSkill

- ☐ Implement `save_contact_segment_score` tool
- ☐ Implement score calculation helpers
- ☐ Write unit tests

TASK-008: Build InteractionLogSkill

- ☐ Implement `log_interaction` tool
- ☐ Implement `get_interaction_history` tool
- ☐ Write unit tests

TASK-009: Build EmbeddingSkill

- ☐ Implement `generate_and_store` tool
- ☐ Integrate OpenAI embedding API

- ☐ Store embeddings in Redis
- ☐ Write unit tests

EPIC 3: Agent Development

TASK-010: Build ContactEnrichmentAgent

- ☐ Implement agent with Claude SDK
- ☐ Integrate ContactQuerySkill, EmbeddingSkill
- ☐ Build context for Claude API calls
- ☐ Parse and store AI insights
- ☐ Write integration tests
- ☐ Create API endpoint `/contacts/{id}/enrich`

TASK-011: Build SegmentCalculatorAgent

- ☐ Implement agent with Claude SDK
- ☐ Integrate SegmentationLogicSkill, ScoringSkill
- ☐ Apply user-defined criteria
- ☐ Store scores in database
- ☐ Write integration tests
- ☐ Create API endpoint `/contacts/{id}/calculate-scores`

TASK-012: Build InsightValidatorAgent

- ☐ Implement agent logic
- ☐ Integrate FeedbackCaptureSkill
- ☐ Move insights between ai_insights ↔ confirmed_facts
- ☐ Track accuracy
- ☐ Write integration tests
- ☐ Create API endpoint `/contacts/{id}/insights/validate`

TASK-013: Build RelationshipScorerAgent

- ☐ Implement scoring algorithm
- ☐ Integrate InteractionLogSkill
- ☐ Calculate relationship strength
- ☐ Determine relationship health
- ☐ Write integration tests
- ☐ Create background job (nightly)

TASK-014: Build NetworkAnalyzerAgent

- ☐ Implement warm path finding
- ☐ Integrate LinkedIn API (if available)
- ☐ Calculate intro success probability
- ☐ Generate intro templates
- ☐ Write integration tests
- ☐ Create API endpoint `/contacts/{id}/warm-paths`

TASK-015: Build CampaignIntelligenceAgent

- ☐ Implement campaign analysis
- ☐ Calculate metrics (open, reply, meeting rates)
- ☐ Segment performance breakdown
- ☐ Generate recommendations
- ☐ Write integration tests
- ☐ Create API endpoint `/campaigns/{id}/analyze`

EPIC 4: API Development

TASK-016: Segmentation Management APIs

- ☐ `POST /segmentations` - Create segment
- ☐ `GET /segmentations` - List segments
- ☐ `PUT /segmentations/{id}/update-criteria` - Update criteria
- ☐ `DELETE /segmentations/{id}` - Delete segment

TASK-017: User Feedback APIs

- ☐ `POST /contacts/{id}/adjust-score` - Adjust score
- ☐ `POST /contacts/{id}/add-fact` - Add custom fact
- ☐ `GET /feedback/summary` - Get feedback summary

TASK-018: Search APIs

- ☐ `POST /search/contacts` - Semantic search
- ☐ `POST /contacts/filter` - Advanced filtering

EPIC 5: Frontend Integration

TASK-019: Segment Management UI

- ☐ Create segment builder interface
- ☐ Filter criteria editor (drag-and-drop)
- ☐ Scoring rules editor
- ☐ Preview matching contacts
- ☐ Save and activate segment

TASK-020: Contact Profile Enhancements

- ☐ Display AI insights with confidence scores
- ☐ "Confirm" / "Reject" buttons for insights
- ☐ "Add Fact" modal
- ☐ "Adjust Score" slider
- ☐ Show confirmed facts separately from AI insights

TASK-021: Feedback Dashboard

- ☐ Show validation accuracy over time
- ☐ Display most adjusted scores
- ☐ Common patterns in user feedback